



Figure1: Raspberry Pi

## Performing an Arch Linux system update

The packages in an Arch Linux distribution get updated continuously—you can always bring the system up-to-date by executing the following command:

```
pacman -Syu
```

'pacman' is the Arch Linux package installer (similar to 'apt' on Debian/Ubuntu). It is recommended that you perform this step and reboot Raspberry Pi.

## Setting up X Windows

Setting up a basic graphical user interface (with X Windows) is simple. Just use 'pacman' to install some packages, as follows:

```
pacman -S xorg-server xorg-apps xorg-
xinit xorg-twm xterm xf86-video-fbdev
You can run X by typing 'startx' at the command prompt.
```

## Installing Python

Let's use Python to write our demo app and install it using 'pacman', as shown below:

```
pacman -S python2
```

You may additionally install package 'python2-pyserial' if you plan to do some serial communication.

## How to write a 'blinking LED' program

**WARNING:** Interfacing electronic circuits to Raspberry Pi might damage the board if not done correctly.

The GPIO pins of a processor are useful for controlling external electronic circuitry. One of the simplest things you can do with these pins is write code, which will output a logic high (3.3V) or low (0V) on them. Raspberry Pi has a set of 26 pins (arranged as a 2x13 strip), which bring out

a few of the GPIO pins of the processor on board; you can identify these pins by looking for label 'P1' on the board. The pins on this connector are numbered P1-01, P1-02, P1-03 and so on. Check out [http://elinux.org/RPi\\_Low-level\\_peripherals](http://elinux.org/RPi_Low-level_peripherals) for a diagram of this connector and details of the I/O pins. (You will note that certain pins are labelled as 'do not connect'. These pins should not be used for interfacing.)

There are many ways in which you can output voltages on these pins. One of the easiest ways is to use a Python library. The Raspberry Pi Python GPIO access library can be downloaded from <http://pypi.python.org/pypi/RPi.GPIO>. It is available as a tar file, RPi.GPIO-0.2.0.tar.gz. (Don't worry if the version number you see on the website is different from this; you might be getting a newer version.) Installing the module is simple; untar it and run the set-up script (as the super user), as follows:

```
tar xvf RPi.GPIO-0.2.0.tar.gz
cd Rpi.GPIO-0.2.0
python2 setup.py install
```

Connect a red LED in series with a 1-kilo-ohm resistor between pins P1-08 (GPIO pin) and P1-06 (GND) and run the following program:

```
import RPi.GPIO as GPIO
import time

GPIO.setup(8, GPIO.OUT)
while True:
    GPIO.output(8, True) # LED ON
    time.sleep(1)
    GPIO.output(8, False) # LED OFF
    time.sleep(1)
```

Configure pin P1-08 as an output pin using GPIO.setup and write high/low values to it using the function GPIO.output.

Internally, the Python GPIO library uses special files under '/sys' directory to access the I/O pins. The disadvantage with this method is that you really can't perform timing-sensitive operations this way.

## Streaming video from a webcam

So what role can Raspberry Pi play in the design of your embedded system?

If your application needs fast media processing and networking, you can replace your PC with Raspberry Pi, which consumes considerably less power (and comes in a much smaller form factor—the size of a credit card). If you need to do timing-sensitive operations (like measuring the time delay between low to high transitions of an I/O pin to microsecond-level precision), you can use something like an Arduino and interface it with Raspberry Pi.

Let us, for example, assume that you wish to capture the video stream from your webcam and make it available over